

Improving the Masked Division for the FALCON Signature

Pierre-Augustin Berthet^{1,2}[0009–0005–5065–2730] and Cédric
Tavernier¹[0009–0007–5224–492X]

¹ Hensoldt France SAS, Plaisir, France

² LTCI, Télécom Paris, Institut Polytechnique de Paris, Palaiseau, France
`berthet@telecom-paris.fr`

Abstract. FALCON is a post-quantum signature selected by the National Institute of Standards and Technology (NIST). Although its side-channel resilience has been studied and a masking countermeasure proposed, the masked division is a major performance bottleneck. This work proposes a different approach to the masked FALCON division. We use the Newton-Raphson method and a convergent sequence to approximate the inversion. The first term of the sequence is evaluated using pre-calculated second-order minimax polynomials. As a consequence, computing the inverse only requires 5 masked additions and 8 masked multiplications, compared to the roughly 55 masked additions required by the previous state-of-the-art. Formal security proofs using the MIMO-SNI criteria are also provided.

Keywords: Post-Quantum Cryptography · FALCON · Side-Channel Analysis · Masking · MIMO-SNI.

1 Introduction

Future quantum technologies will solve the Discrete Logarithm Problem (DLP) thanks to the Shor algorithm [1] in practicable times. The National Institute of Standards and Technology (NIST) launched a competition [2] to select the next post-quantum cryptography standards. Among them is FALCON [3], a lattice-based signature.

Another threat is Side-Channel Analysis (SCA), first published by Paul Kocher [4]. An opponent with physical access to the device under attack can correlate the device’s environmental parameters (power consumption, execution timing, etc.) with its sensitive data. It is important to study the resilience of FALCON against such attacks.

1.1 Related works

Several generic countermeasures were applied to defend FALCON against a side-channel opponent. Chen and Chen [5] proposed a masking methodology for the addition and multiplication of floating numbers. Karabulut and Aysu [6] did a

similar work on a hardware platform, specifically on multiplication. Berthet et al. [7] completed Chen and Chen’s [5] design and provided a masking methodology for the floor function and the division, as well as an estimate of the performances of a completely masked FALCON on a laptop computer.

1.2 Contribution

This paper proposes a different method for computing a masked division within FALCON. This design relies on the Newton-Raphson method to find a root for a smooth function, in this case the inversion. The first term of the sequence used in the Newton-Raphson method is evaluated in a masked way thanks to second-order minimax polynomials. Formal proofs of security relying on the MIMO-SNI criteria are provided.

Section 2 introduces the background and notations. Section 3 presents a new masked design for the division in FALCON. Section 4 analyses its formal security. Section 5 concludes the paper.

2 Preliminaries

2.1 Notations

We denote by (α_i) a secret sharing of a sensitive datum α . $\alpha_k, k \in \mathbb{N}$, refers to the k -ith share of the sharing. An exception is made for the specific case of a_0, a_n, a_{n+1} , and b_0, b_n, b_{n+1} , which are the 0-ith, n -ith and $n + 1$ -ith terms of convergent sequences.

The constant values $\sigma_{min} = 1.277833697$ and $\sigma_{max} = 1.8205$ are parameters of the FALCON-512 signature. For FALCON-1024, $\sigma_{min} = 1.298280334$.

Notations in Algorithm 1 are from the FALCON reference paper [3].

2.2 Masking

Overall idea Among the generic countermeasures most studied against SCA is *masking* [8]. The sensitive datum is separated into random *shares* to decorrelate it from its leakage. The shares form a *sharing* of the secret. Each share is processed independently from the others. Recombining the shares after sensitive computations are performed ensures correctness.

Formal proofs To verify the effectiveness of a masked gadget, we rely on formal proofs. Ishai et al. [9] introduced the concept of t -probing security, formalising the resilience of a device against a side-channel opponent with measurement probes. However, the composability of t -probing gadgets is not necessarily secure. To address this issue, Barthe et al. [10] proposed the concept of t -(Strong-)Non-Interference (t -NI, t -SNI):

Definition 1 (t -Non Interference (t -NI) security [10]). A gadget is said t -Non Interference (t -NI) secure if every set of t intermediate values can be simulated by no more than t shares of each of its inputs.

However, t -NI gadgets composition does not imply t -NI security. Barthe et al. [10] propose a stronger definition for this:

Definition 2 (t -Strong Non Interference (t -SNI) security [10]). A gadget is said t -Strong Non-Interference (t -SNI) secure if for every set of t_I of internal intermediate values and t_O of its output shares with $t_I + t_O \leq t$, they can be simulated by no more than t_I shares of each of its inputs.

Although the composition of t -SNI gadgets is t -SNI, it only applies to single-output gadgets. To prove the security of Multiple Inputs - Multiple Outputs (MIMO) gadgets, Cassiers and Standaert [11] introduced the t -MIMO-SNI model:

Definition 3. (t -MIMO-SNI security [11]). Let $\mathcal{O}_i$ be a set of shares indices for $i = 0, \dots, d-1$. A gadget is t -MIMO-SNI if and only if for any set $\mathcal{I}$ of t_1 internal probes and any sets $\mathcal{O}_i$ such that there exists a t_2 that satisfies $t_1 + t_2 \leq t$ and $|\mathcal{O}_i| \leq t_2$ for $i = 0, \dots, d-1$, the sets of probes $\mathcal{I} \cup y_{\mathcal{O}_0,0} \cup \dots \cup y_{\mathcal{O}_{d-1},d-1}$ can be simulated with at most t_1 input shares.

As we use composition proofs to formally verify the security of our design, a list of the sub-gadgets used for the masked division is the following:

- SECFPRMUL, which performs a masked multiplication on the floating-point masking of Chen and Chen [5],
- SECFPRADD, which performs a masked addition on the floating-point masking of Chen and Chen [5].

Both gadgets have been proved t -SNI secure by Chen and Chen [5].

2.3 FALCON and division

FALCON [3], or FN-DSA, is a lattice-based signature selected by the NIST for standardisation. It relies on the NTRU and SIS problems over the GPV framework [12].

Its peculiarity is to use a Gaussian sampler and floating-point arithmetic. The sampler, named SamplerZ, is described in Algorithm 1. The function BASESAMPLER outputs an integer between 0 and 18. The function UNIFORMBITS(8) outputs 8 random bits uniformly. The function BEREXP(x, ccs) outputs a single bit 1 with probability $ccs \cdot \exp(-x)$.

The calculations addressed in this work are the division by σ' in the Gaussian sampler, at line 2 of Algorithm 1 and in the **while** loop at line 7 of Algorithm 1. Although there are two divisions, as they use the same datum, we can perform them once and simply refresh the result for its second use.

Algorithm 1: SamplerZ(μ, σ') [3]

Data: floating-point values $\mu, \sigma' \in \mathbb{R}$ such that $\sigma' \in [\sigma_{\min}, \sigma_{\max}]$
Result: $z \in \mathbb{Z}$ sampled from a distribution very close to $D_{\mathbb{Z}, \mu, \sigma'}$

```

1  $r \leftarrow \mu - \lfloor \mu \rfloor$ ;
2  $ccs \leftarrow \sigma_{\min} / \sigma'$ ;
3 while 1 do
4    $z_0 \leftarrow \text{BASESAMPLER}()$ ;
5    $b \leftarrow \text{UNIFORMBITS}(8) \text{ AND } 0x1$ ;
6    $z \leftarrow b + (2 \cdot b - 1)z_0$ ;
7    $x \leftarrow \frac{(z-r)^2}{2\sigma'^2} - \frac{z_0^2}{2\sigma_{\max}^2}$ ;
8   if  $\text{BEREXP}(x, ccs) = 1$  then
9     return  $z + \lfloor \mu \rfloor$ ;
```

3 FALCON masked division

Instead of computing a masked division, we calculate a masked inversion and then perform a masked multiplication. In [7], the masked inversion used the naive Euclidean approach. This leads to a high complexity in terms of masked additions and multiplications, to a point where the inversion takes more than 50% of the overall cost of one call to the Gaussian sampler. FALCON performs either 1024 or 2048 calls to this sampler in one rejection loop, hence reducing the cost of the inversion improves the overall performance of the masked signature significantly. We use a different approach based on the Newton-Raphson method.

3.1 Newton-Raphson Method

To find a root β of a smooth function $f : \mathbb{C} \rightarrow \mathbb{C}$ such as the derivative of f in β is not zero, we can use the iterative scheme

$$a_{n+1} = a_n - \frac{f(a_n)}{f'(a_n)}. \quad (1)$$

If the first term a_0 is chosen close enough to β , this scheme will converge quadratically to this root.

To compute the inverse, there are several possibilities in the choice of f . Several works [13, 14] propose the same sequence for the computation of the inverse. It is described in Equation (2):

$$a_{n+1} = a_n(2 - a_n x). \quad (2)$$

However, the sequence converges efficiently only if the first term a_0 is a good approximation of the final result. As we perform this inversion in a masked manner, we have to compute this first approximation using the knowledge we have on the masked value σ' without revealing its true value.

In the FALCON Gaussian sampler described in Algorithm 1, we have $\sigma' \in [\sigma_{\min}; \sigma_{\max}]$. To approximate the first term of the sequence, we use a second-order minimax polynomial [15]. Minimax polynomials provide a good approximation of a function as they are computed so that the delta between the polynomial and the function it approximates is the lowest on average. We have the following approximation polynomial for the parameter set used in FALCON-512:

$$t(x) = 0.27964057x^2 - 1.29293999x + 1.97721479. \quad (3)$$

For the parameter set used in FALCON-1024, the values are slightly different:

$$t(x) = 0.27324973x^2 - 1.27229618x + 1.96069036. \quad (4)$$

The evaluation of t in σ' can be performed masked and gives us a good first approximation of the inverse of σ' without revealing its value. The first term of the Newton-Raphson sequence is thus $a_0 = t(\sigma')$.

The number of iterations of the sequence to perform depends on the precision required by FALCON. The signature is tailored around the double precision for floating points, i.e. the tolerated margin of error is smaller than $1.11 * 10^{-16}$. To evaluate the number of iterations, we define the sequence $b_n = a_n - 1/x$. We have the following result:

$$\begin{aligned} lCla_{n+1} &= a_n(2 - a_nx) \\ &= (b_n + 1/x)(2 - (b_n + 1/x)x) \\ &= (b_n + 1/x)(1 - b_nx) \\ &= 1/x - b_n^2x \\ b_{n+1} &= -b_n^2x \\ b_n &= (-1)^n b_0^{2^n} x^{2^n - 1} \end{aligned}$$

As $b_0 = a_0 - 1/x = t(x) - 1/x$, we can rewrite the sequence b_n as follows:

$$b_n = (-1)^n x^{2^n - 1} (t(x) - 1/x)^{2^n} \quad (5)$$

A study of the number of iterations required by FALCON-512 is provided in Table 1, and by FALCON-1024 in Table 2. Both tables were obtained with a computational model like Wolfram Alpha. They highlight the maximum value of $|b_n|$ for several values of n . The optimum number of iterations for the sequence from Equation (2) in the case of FALCON is 3, for both security levels.

Table 1. Maxima of the values of $|b_n|$ for $n \in \llbracket 1; 5 \rrbracket$ and $b_0 = t(x)$, with $x \in [\sigma_{\min}, \sigma_{\max}]$, for FALCON 512

Number of terms n	1	2	3	4	5
Magnitude of $ b_n $	10^{-6}	10^{-11}	10^{-22}	10^{-45}	10^{-90}

Table 2. Maxima of the values of $|b_n|$ for $n \in \llbracket 1; 5 \rrbracket$ and $b_0 = t(x)$, with $x \in [\sigma_{min}, \sigma_{max}]$, for FALCON 1024

Number of terms n	1	2	3	4	5
Magnitude of $ b_n $	10^{-6}	10^{-11}	10^{-23}	10^{-46}	10^{-92}

3.2 Masked Algorithm

Given the methods described in the previous section, a masked inversion using the sequence based on Equation (2) is described in Algorithm 2. The algorithmic variables $(csts_i)$, $(cstm_i)$ and $(csta_i)$ refer to 64-bit Boolean sharings of the constants used in the second-order minimax polynomial used to approximate the first term of the Newton-Raphson sequence. $(csts_i)$ refers to the coefficient of degree 2, $(cstm_i)$ of degree 1 and $(csta_i)$ of degree 0, i.e. coefficients of Equations 3 and 4. A 64-bit Boolean sharing (α_i) of a constant floating value α is trivially $(\alpha_i) = (\alpha, 0, \dots, 0)$, with α encoded in *Binary64* format.

Algorithm 2: Masked Inversion for FALCON

Data: 64-bit Boolean sharing (x_i) of the secret x and an integer n number of iterations.

Result: 64-bit Boolean sharing (out_i) for value $1/x$.

```

1  $(cst_i) \leftarrow (0x4000000000000000, \dots, 0)$  // cst = 2
2  $(tsq_i) \leftarrow \text{SECFPRMUL}((x_i), (csts_i))$ 
3  $(tmp_i) \leftarrow \text{SECFPRADD}((tsq_i), (cstm_i))$ 
4  $(tmp_i) \leftarrow \text{SECFPRMUL}((x_i), (tmp_i))$ 
5  $(tmp_i) \leftarrow \text{SECFPRADD}((tmp_i), (csta_i))$ 
6 for  $i$  from 1 to  $n$  do
7    $(xa_i) \leftarrow \text{SECFPRMUL}((x_i), (tmp_i))$ 
8    $(xa_0) \leftarrow (xa_0) \text{ XOR } (1 \ll 63)$  // Equivalent to  $(xa_i)$  times -1
9    $(xa_i) \leftarrow \text{SECFPRADD}((xa_i), (cst_i))$ 
10   $(tmp_i) \leftarrow \text{SECFPRMUL}((tmp_i), (xa_i))$ 
11 return  $(out_i) = (tmp_i)$ 

```

The inversion described in Algorithm 2 has a complexity of $2 + 2n$ masked multiplications and $2 + n$ masked additions, for a total of $4 + 3n$ masked operations. We take $n = 3$ and compute the masked inversion in FALCON in 13 masked operations, that is, 8 masked floating-point multiplications and 5 masked floating-point additions. This beats the 55 masked additions required by SECFPRINV in the previous state-of-the-art [7].

4 Security

To formally verify the security of our design, we use the methodology described by Cassiers and Standaert [11]. They model a gadget as a Directed Acyclic Graph (DAG), or computation graph, where each vertex is a t -NI secure subgadget and the edges are the wires connecting the subgadgets to form the main gadget. Every subgadget has one output. A duplication vertex, denoted $Split_j$ is used if a datum is reused by j subgadgets. If a subgadget is known to be t -SNI secure, it is shown as a t -NI secure subgadget followed by a t -SNI Refresh vertex.

This graph model highlights how the sensitive datum's information propagates through the gadget and thus which information a specific probe placement can reveal. Cassiers and Standaert [11] define the Simplified Computation Graph as follows:

Definition 4 (Simplified Computation Graph (SCG), [11]). *The simplification of the computation graph G is the graph obtained from G by removing all the t -SNI Refresh vertices and their incident edges.*

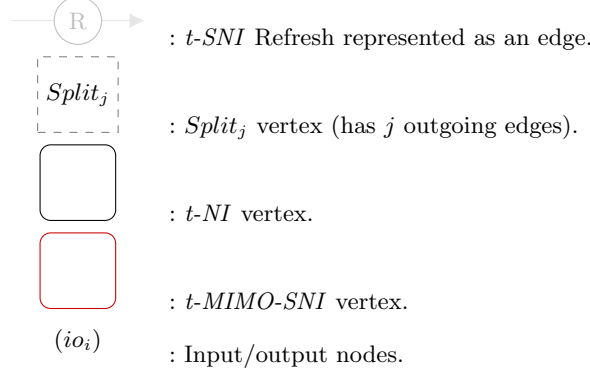
According to [11] (Proposition 7, page 2549), to prove the t -NI security of a gadget, it is sufficient to verify that its SCG is a Single Path - NI Built gadget (SP-NIB):

Definition 5 (Single Path - NI Built gadget (SP-NIB), [11]). *A composite gadget G is SP-NIB if it is implemented with only NI gadgets and SNI refreshes, and if for any pair of vertices u, v in the corresponding simplified computation graph there exists at most one path from u to v .*

The conditions required for an SCG to verify t -MIMO-SNI security are as follows:

Proposition 1 ([7, 11]). *A composite gadget G is t -MIMO-SNI if it satisfies the three following conditions. (i) G is SP-NIB. (ii) For any pair of output nodes u_1, u_2 there is no node v such that there is a path from v to u_1 and a path from v to u_2 . (iii) For any pair of input nodes u_1, u_2 there is no node v such that there is a path from u_1 to v and a path from u_2 to v . (iv) There is no path from any input node to an output node.*

To prove the t -SNI security of our inversion, we perform a composition proof using the t -MIMO-SNI criteria and Proposition 1 for the sub-gadgets of Algorithm 2. We use the legend described in Table 3 for the SCG used in the proofs. The t -SNI Refresh vertices are depicted as grey edges for clarity. They are not considered edges or vertices in the SCG when performing the proofs.

Table 3. SCG legend

Lemma 1. *The For loop from Algorithm 2 is t -MIMO-SNI secure when we take into account that the input of the first iteration is fed by a t -SNI gadget.*

Proof. The For loop is depicted in Figure 1. We choose to depict the edge from the input (tmp_i) to its $Split_2$ node as a t -SNI Refresh. The justification is twofold:

- The first iteration of the For Loop takes as input (tmp_i) which is outputted by a *SecFprAdd* gadget. This is a t -SNI gadget, thus its representation in the SCG is a t -NI node followed by a t -SNI Refresh. This Refresh is thus represented in Figure 1 when we consider the first iteration of the For loop.
- The output (tmp_i) of the For loop is preceded by a t -SNI Refresh. As this output will serve as an input to the next iteration of the For loop, we can once again depict the edge from the input (tmp_i) to $Split_2$ as a t -SNI Refresh. This Refresh after the input thus represents the Refresh before the output from the previous iteration of the For loop.

This being clarified, we now verify that the conditions of Proposition 1 are met by the SCG in Figure 1.

- (i) For every couple of nodes u, v , there is at most one path from u to v . The SCG is thus SP-NIB and we have t -NI security.
- (ii) As there is only one output, the second condition of Proposition 1 is verified.
- (iii) As explained previously, the input (tmp_i) is always refreshed and thus not “connected” to the remainder of the graph. As the input (in_i) is the only other input, the third condition of Proposition 1 is verified.
- (iv) As there is a t -SNI Refresh before the only output of the For loop, there is no path from any input to the output. The final condition of Proposition 1 is verified.

Hence, according to Proposition 1, each iteration of the For loop from Algorithm 2 is t -MIMO-SNI secure, when taking into account that the input of the first iteration is fed by a t -SNI gadget. By composition of t -MIMO-SNI gadgets, the entire For loop is t -MIMO-SNI secure.

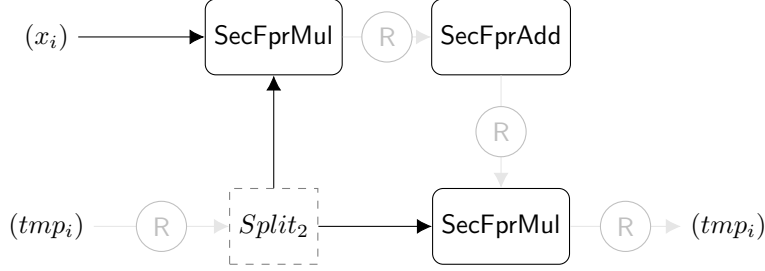


Fig. 1. SCG of the For loop of Algorithm 2

Theorem 1. *The inversion described in Algorithm 2 is t -SNI secure.*

Proof. The SCG of the algorithm is shown in Figure 2. For clarity, the n edges from node $Split_{n+2}$ to each iteration of the For loop are represented as one edge. As the For loop is t -MIMO-SNI secure according to Lemma 1. The SCG verifies the conditions of Proposition 1:

- (i) The SCG depicted in Figure 2 is SP-NIB.
- (ii) There is only one output.
- (iii) There is only one input.
- (iv) As the output is preceded by a t -SNI Refresh, there is no path in the SCG between the input and the output.

A single-input single-output t -MIMO-SNI gadget is a t -SNI gadget. The inversion described in Algorithm 2 is t -SNI secure.

5 Conclusion

In this paper, we present a different methodology to perform the masked division within the FALCON post-quantum signature compared to the previous state-of-the-art masked implementation of FALCON [7]. We use the Newton-Raphson method and a convergent sequence to approximate the value of the inverse of a masked value. The first term of the sequence is approximated in a masked way thanks to the use of second-order minimax polynomials [15]. Formal proofs in the t -MIMO-SNI model are provided and the inversion is proved t -SNI secure.

The next evolution of this work is to port the implementation on an embedded target, either microcontroller, FPGA or ASIC, to perform some experimental validation of the security of the design and prepare it for deployment.

References

1. P. W. Shor, “Algorithms for quantum computation: discrete logarithms and factoring,” *Proceedings 35th Annual Symposium on Foundations of Computer Science*, Santa Fe, NM, USA, 1994, pp. 124-134, doi: 10.1109/SFCS.1994.365700 .

10. G. Barthe et al., “Strong non-interference and type-directed higher-order masking,” In Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS ’16). Association for Computing Machinery, New York, NY, USA, 116–129, doi: 10.1145/2976749.2978427
11. G. Cassiers and F. -X. Standaert, “Trivially and efficiently composing masked gadgets With probe isolating non-interference,” in IEEE Transactions on Information Forensics and Security, vol. 15, pp. 2542-2555, 2020, doi: 10.1109/TIFS.2020.2971153
12. C. Gentry, C. Peikert and V. Vaikuntanathan, “Trapdoors for hard lattices and new cryptographic constructions,” in Proceedings of the fortieth annual ACM symposium on Theory of computing (STOC ’08), Association for Computing Machinery, New York, USA, 2008, pp. 197-206, doi: 10.1145/1374376.1374407
13. P. Soderquist and M. Leiser, “Division and square root: choosing the right implementation,” in IEEE Micro, vol. 17, no. 4, pp. 56-66, July-Aug. 1997, doi: 10.1109/40.612224
14. J. D. Blanchard, and M. Chamberland, “Newton’s Method Without Division,” The American Mathematical Monthly, 130(7), pp. 606–617, doi: 10.1080/00029890.2022.2093573
15. M. P. Vestias and H. C. Neto, “Revisiting the Newton-Raphson Iterative Method for Decimal Division,” 2011 21st International Conference on Field Programmable Logic and Applications, Chania, Greece, 2011, pp. 138-143, doi: 10.1109/FPL.2011.33